



**Padrões de Projeto
(Design Patterns)
em Java**

João Paulo A. Almeida
jp Almeida@ieee.org
<http://nemo.inf.ufes.br>

Departamento de Informática
Universidade Federal do Espírito Santo

Design Patterns / Padrões de Projeto



- Um padrão de projeto é uma solução geral para um problema recorrente
- É uma espécie de “gabarito” para como resolver um problema
- Captura boas práticas
- Serve para criar uma linguagem de comunicação
- (veja o livro de Gamma et. al. (GoF Gang of Four) intitulado “Design Patterns”)
- Aqui alguns úteis aplicados a Java...

Separação de Implementação e Interface



- Já vimos este padrão em ação nas collections de Java:
- `interface List<T>`
- `class ArrayList<T> implements List<T>`
- `class LinkedList<T> implements List<T>`
- Neste caso permite abstrair as implementações específicas de List (a dependência só existe na criação)

• Uso:

```
List<String> lista = new LinkedList<String>();
lista.add("Teste");
...
```

Separação de Implementação e Interface



- Às vezes aplicamos o padrão até mesmo para prever alterações futuras, e desacoplar o uso do objeto da implementação

• Exemplo:

```
interface Product { public void getPrice(); }
class ProductImpl implements Product {
    public void getPrice() { ... }
}
```

```
Uso: Product p = new ProductImpl();
p.getPrice();
```

- Permite alteração futura da classe que implementa ...

GoF Patterns



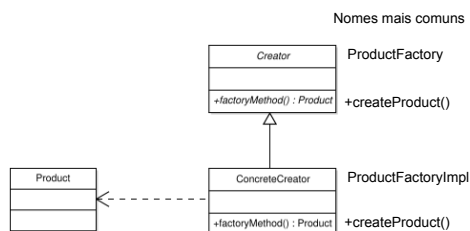
- Creational Patterns
 - Factory
 - Factory with Finder
 - Singleton
- Structural Patterns
 - Decorator
 - Composite
- Behavioural Patterns
 - Iterator
 - Observer / Listener

Factory pattern



- Outro padrão que esconde ainda mais os detalhes de criação (incluindo qual a classe usada)
- Não usamos o operador new, mas pedimos a criação para uma classe especial: a fábrica (Factory)

Factory pattern



Factory: interface

```

interface Product
{
    // qualquer definição do objeto
    bla bla bla
}

class ProductImpl implements Product
{
    // qualquer definição do objeto
    bla bla bla
}

interface ProductFactory
{
    Product createProduct();
}
    
```

Factory: implementação

```

class ProductFactoryImpl implements ProductFactory
{
    public Product createProduct()
    {
        // cria um objeto, decidindo qual a implementação
        return (Product) new ProductImpl();
    }
}
    
```

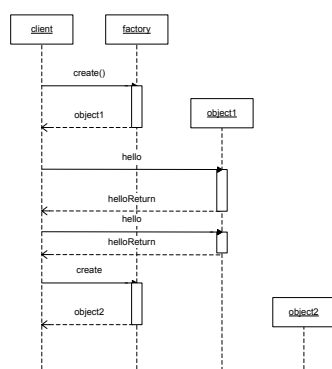
Factory: outra implementação

```

interface ProductFactory
{
    Product createProduct(int productCode);
}

class ProductFactoryImpl implements ProductFactory
{
    public Product createProduct(int productCode)
    {
        // cria um objeto, decidindo qual a implementação
        if (productCode==1)
            return (Product) new Product1Impl();
        else if (productCode==2)
            return (Product) new Product2Impl();
    }
}
    
```

Dinâmica do Factory Pattern



Exemplo em uso

```

public class Conta
{
    ...
}

public class Banco {
    private Set<Conta> contas=new Set<Conta>();

    Conta criaConta() {
        Conta c = new Conta();
        contas.add(c);
        return c;
    }
}
    
```

Factory c/ Finder

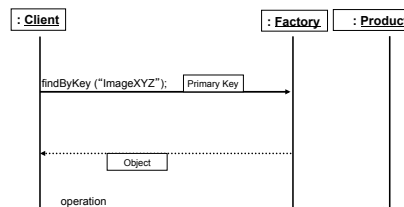
```
interface Product
{
    bla bla bla
}
```

```
interface ProductFactory
{
    Product createProduct(keytype key);
    Product findByKey(keytype key);
}
```

- // exercício: que exceções seriam relevantes?
- // sugerir implementação



Factory/Finder Pattern Dynamics



Singleton object

- Acesso a objeto global
- Ao invés de usar métodos e atributos estáticos, que seria uma outra forma de fazer isto



Singleton

```
public class SingleObject {
    //create an object of SingleObject
    private static SingleObject instance = new SingleObject();
    //make the constructor private so that this class cannot be instantiated
    private SingleObject(){}

    //Get the only object available
    public static SingleObject getInstance(){
        return instance;
    }

    public void showMessage(){
        System.out.println("Hello World!");
    }
    // métodos aqui
}
```



Singleton a partir de Java 1.6

```
package mypackage;

public enum MyEnumSingleton {
    INSTANCE;

    // métodos aqui
}
```

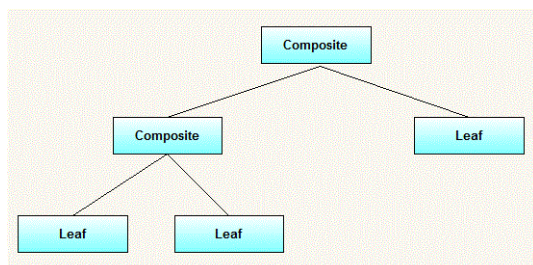


Composite Pattern

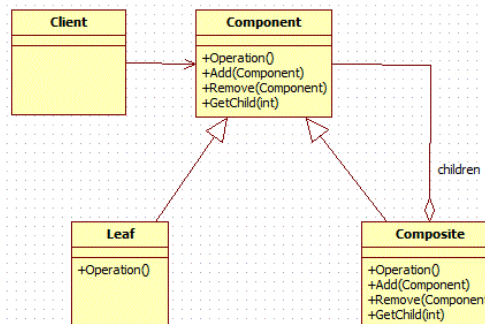
- Normalmente usado para uma árvore de “containment”
 - Um elemento de um mesmo tipo “contém” elementos do mesmo tipo
 - Exemplo:
- ```
class Pasta
{
 List<Pasta> subpastas=new LinkedList<Pasta>();
 List<Arquivo> arquivos=new LinkedList<Arquivo>();
}
```
- // a implementação permitiria laços apesar de não ser a ideia



## Composite Pattern



## Composite Pattern



## Composite Pattern (degenerado)



```

class Funcionario
{
 String nome; double salario;
 List<Funcionario> subordinados=
 new LinkedList<Funcionario>();
}

```

## Decorator



- Também conhecido como Wrapper
- Permite estender funcionalidade de uma classe existente de uma forma diferente da herança
- Na verdade permite em tempo de execução estender um objeto existente
- Já vimos uso no caso dos Readers:
- `BufferedReader` é um Decorator de `FileReader`, adicionando métodos
- `BufferedReader in = new BufferedReader(new FileReader("foo.in"));`
- Passamos a usar o `BufferedReader`, que "encapsula" o `FileReader`, adicionando `mark` e `reset`

## Iterator (java.util.Iterator)



```

interface Iterator<E>
{
 // Returns true if the iteration has more elements
 boolean hasNext();
 // Returns the next element in the iteration
 // pode jogar NoSuchElementException
 E next();
}

```

Incorporado nas coleções de Java, mas também em `Scanner` (na verdade `Scanner` implements `Iterator<String>`)

## Scanner é um Iterator&lt;String&gt;



```

java.util.Scanner s = new java.util.Scanner("teste;;teste2");
s.useDelimiter(";");
System.out.println(s.hasNext());
System.out.println "[" + s.next() + "]";
System.out.println(s.hasNext());
System.out.println "[" + s.next() + "]";
System.out.println(s.hasNext());
System.out.println "[" + s.next() + "]";
System.out.println(s.hasNext());

```

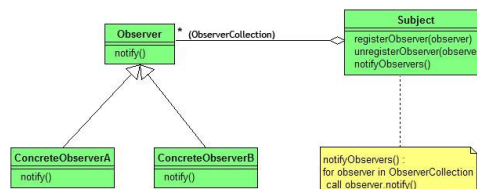
### “foreach” usa Iterator...

```
for (Pessoa p : listapessoas)
{
 System.out.println(p.getNome());
}
```

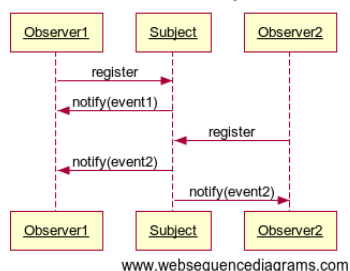
Possível para qualquer classe que implemente Iterable<T>

```
public interface Iterable<T>
{
 Iterator<T> iterator();
}
```

### Observer ou Listener



### Dynamics of the Observer/Listener pattern



### Listener / Observer

```
interface Listener
```

```
{
 public void notify();
}
```

```
public Observer implements Listener
```

```
{
 public void notify()
 {
 System.out.println("Conta criada");
 }
}
```

### Listener / Observer

```
interface Listener
```

```
{
 public void notify();
}
```

```
public Observer implements Listener
```

```
{
 public void notify()
 {
 System.out.println("Conta criada");
 }
}
```

### Exemplo em uso

```
public class Banco {
 private Set<Conta> contas=new Set<Conta>();
 private Set<Listener> listeners=new Set<Listener>();
 public addListener(Listener listener) {
 listeners.add(listener);
 }
 public removeListener(Listener listener) {
 listener.remove(listener);
 }
 public Conta criaConta() {
 Conta c = new Conta();
 contas.add(c);
 for (Listener listener : listeners) { listener.notify(); }
 return c;
 }
}
```

### Exemplo em uso (Listener)



```
public static void main()
{
 Banco b = new Banco();
 Observer o = new Observer();
 b.addListener(o);
 b.criaConta();
}
```